
Locality as the Bottleneck: Topology-Preserving Level Repair with Neural Cellular Automata

September 4, 2026

Niccolò Pozio

Abstract

We study if a Neural Cellular Automaton, trained only to reconstruct real rooms from the dungeons of the first *The Legend of Zelda*, can restore the playability of a damaged room, that is the topological connectivity between its accesses and its main walkable area. Connectivity never appears in the cost function, and we evaluate the repair with a criterion based on pathfinding rather than on tile accuracy. With the same number of destroyed cells the success rate varies $6.7\times$ depending on the target of the damage, and drops to 0.100 when we delete an access, while the rooms are still 98.9% correct tile-wise. Capacity, unroll length, perception and explicit topological supervision leave this failure unchanged, while a single global pass of a U-Net repairs accesses $5.1\times$ better, isolating locality as the bottleneck. Averaging one hidden channel over the grid, at the same parameter count, recovers part of the gap.

1. Introduction

In a neural cellular automaton (NCA) (Mordvintsev et al., 2020) a grid of cells is updated by a small convolutional network, learned rather than fixed, which reads only the immediate neighbourhood. A common use of NCAs is regeneration, where the model reconstructs a target from a damaged version of it. We move this task to videogame levels, where accuracy is not enough, because levels have an underlying logical structure that must be preserved, a repaired room can achieve 99% tile accuracy and still be unplayable. NCAs have already been used to generate playable levels, with solvability as an optimised objective (Earle et al., 2022), and level repair has been addressed with autoencoders (Jain et al., 2016), mixed-integer programming (Zhang et al., 2020) and pathfinding agents, but always with global models, to the best of our knowledge no

NCA has been evaluated on repairing real game levels. A further point of our work is that connectivity never appears in the cost function, we want to see if the model preserves it without being directed towards it. Models are trained on real dungeon rooms, damaged in many ways, and we evaluate the repaired rooms on traversability. Code and weights are available at <https://github.com/NicoPozio/zelda-nca>.

2. Related Works

NCAs were introduced as a model of morphogenesis, where a local rule grows a target from a seed and regenerates it after damage (Mordvintsev et al., 2020), and were later extended to persistent textures (Niklasson et al., 2021), discrete tasks (Randazzo et al., 2020), generative modelling (Palm et al., 2022) and 3D structures (Sudhakaran et al., 2021). In all of them regeneration is measured by fidelity, never by a functionality.

In games, NCAs have been evolved into level generators (Earle et al., 2022), pathfinding itself has been implemented as an NCA (Earle et al., 2023), and levels have been generated by learning to invert a trace of progressive destruction (Siper et al., 2022). The data come from the VGLC (Summerville et al., 2016), which has been used before to learn the topology of Zelda dungeons at the graph level (Summerville et al., 2015), whereas we work inside a single room. The comparison model is a U-Net (Ronneberger et al., 2015). The intersection, local regeneration of real levels evaluated by playability, appears to be unexplored.

3. Methodology

Dataset We extract 236 rooms from the 9 dungeon files of the first quest in the VGLC; 136 are unique up to symmetry and we keep only those, since the game reuses layouts across dungeons. We split them into 96 training rooms and 20 each for validation and test, augmenting only the training set with the four shape-preserving symmetries, so that no symmetric version of a training room reaches the held-out sets. An 11×16 room is one-hot encoded over

Email: Niccolò Pozio <pozio.2085512@studenti.uniroma1.it>.

10 tile categories, to which we append 12 hidden channels initialised to zero.

Metric: The VGLC does not annotate breakable walls or movable blocks, so an absolute reachability criterion would fail on rooms that are playable in the game. We therefore define our regeneration success rate (RSR) relative to the pristine room, as the fraction of rooms where the main walkable component keeps the same cells and the accesses are the same.

Damage taxonomy: Training damage is stochastic, the erasure of a contiguous patch, or the flipping of scattered tiles to a wrong category. Evaluation damage is targeted and never appears in training, so that repairing it tests generalisation. We delete the accesses of a room (B1), or the floor in front of an access, leaving it unreachable (B3). A matched-extent control (A3) destroys as many walkable cells as B1, chosen at random, so that target and extent vary independently.

Models: The NCA has a 22-channel state per cell and perceives its 3×3 neighbourhood through a depthwise convolution with three fixed filters, identity and the two Sobel operators. A two-layer MLP (1×1 convolutions, 128 units) outputs a residual increment, applied under a Bernoulli(0.5) per-cell mask, for a total of 11,414 parameters. A variant replaces one hidden channel, after every step, by its spatial mean: each cell still reads only its neighbourhood but also receives a scalar summary of the global state, at the same parameter count. The U-Net probes what locality costs: two pooling stages give every bottleneck cell a receptive field larger than the room itself, whereas an NCA cell extends its reach by one cell per step. Its 473,974 parameters bound what capacity alone could achieve.

Protocol: We train through a sample pool of 1024 states, each a room at some point in its evolution. For 8000 iterations, where the loss plateaus, we draw a batch of 8, replace the worst state with a clean room, damage half of the batch, run the model on each state for a number of steps drawn in [64, 96], and write the evolved states back into the pool. The loss is a cross-entropy over the visible channels against the pristine room, optimised with Adam and gradient clipping. We train one NCA that never sees damage and one that does, then vary a single hyperparameter at a time around the latter; the BFS-supervised and the globally pooled models are variants of it that change the loss and the update rule respectively. We have 2 variants of the U-Net, a one-shot and an iterative one. Evaluation is deterministic: the update mask is drawn at every step and we fix its seed. Configurations use three seeds, eight for the main comparisons; ablations are on validation, final comparisons on test.

Table 1. Test-set RSR over training seeds. A0 applies no damage and sets the ceiling; A3, B1 and B3 destroy the same number of cells.

Model	A0 none	A3 random	B1 access	B3 isolation
no-damage NCA	0.967	0.328	0.122	0.189
NCA	0.973	0.482	0.100	0.675
iterative U-Net	0.950	0.817	0.156	0.928
one-shot U-Net	1.000	0.150	0.506	0.222
BFS NCA	0.917	0.578	0.152	0.783
global-channel	0.971	0.268	0.273	0.477

4. Experimental Results

Damage in training produces regeneration: Trained only on undamaged rooms the NCA becomes a stabiliser: it matches the main model on A0 but repairs almost nothing.

Target and extent of the damage: The three damages destroy the same number of cells, yet the RSR varies $6.7\times$, and B3 reaches 69% of the ceiling set by A0. The ordering follows how far the local neighbourhood agrees with the correct answer, an access is surrounded by walls, so locality pushes the model to write a wall there. Tile accuracy does not see this failure, since on B1 the rooms are still 98.9% correct (Appendix C).

Locality is the bottleneck: Hidden channels, unroll length, update probability, MLP width, a Laplacian filter and the BFS supervision all leave access repair unchanged (Appendix B), while a single global pass repairs accesses $5.1\times$ better than 64–96 local steps. But no model is better everywhere: on A3 and B3 the ordering reverses and the one-shot U-Net is the worst of the three.

A scalar summary recovers part of the gap: Averaging one hidden channel over the grid, at the same parameter count, raises access repair from 0.100 to 0.273, at a cost on the other conditions: A3 falls to 0.268 and B3 to 0.477.

5. Conclusion

- The NCA repairs the room only up to a narrow band of damage, RSR falls from 0.358 at 5% of the room destroyed to 0.031 at 20%.
- What influences the repair is not the extent of the damage but its target.
- The limitation is locality, none of the strategies that keep the rule strictly local changed the repair of the accesses, while a single hidden channel averaged over the grid raises it $2.7\times$ and a global pass $5.1\times$.

The main limitation is that the test set is small (20 rooms) and the dataset describes geometry but not affordances.

Statement on the use of AI

Claude was used as a coding and writing assistant. Specifically, it was used to: (i) write draft code from my specifications, which I then reviewed, tested and modified; (ii) debug the training and evaluation pipeline; (iii) produce the explanatory notebook in the repository; and (iv) rework this manuscript for English and for the two-page limit.

The decisions that shape the project are mine: the relative topological criterion, the matched-extent control, the damage taxonomy, the domain rules the metric depends on, such as which tiles count as traversable in which context, and which ablations to run. Every number reported here was produced by runs I executed on Kaggle and checked against the released checkpoints. I understand the codebase and take responsibility for the submitted work.

References

- Earle, S., Snider, J., Fontaine, M. C., Nikolaidis, S., and Togelius, J. Illuminating diverse neural cellular automata for level generation. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, pp. 68–76, 2022. doi: 10.1145/3512290.3528754.
- Earle, S., Yildiz, O., Togelius, J., and Hegde, C. Pathfinding neural cellular automata. *arXiv preprint arXiv:2301.06820*, 2023.
- Jain, R., Isaksen, A., Holmgård, C., and Togelius, J. Autoencoders for level generation, repair, and recognition. In *Proceedings of the ICCG Workshop on Computational Creativity and Games*, 2016.
- Mordvintsev, A., Randazzo, E., Niklasson, E., and Levin, M. Growing neural cellular automata. *Distill*, 2020. doi: 10.23915/distill.00023.
- Niklasson, E., Mordvintsev, A., Randazzo, E., and Levin, M. Self-organising textures. *Distill*, 2021. doi: 10.23915/distill.00027.003.
- Palm, R. B., González-Duque, M., Sudhakaran, S., and Risi, S. Variational neural cellular automata. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2201.12360.
- Randazzo, E., Mordvintsev, A., Niklasson, E., Levin, M., and Greydanus, S. Self-classifying MNIST digits. *Distill*, 2020. doi: 10.23915/distill.00027.002.
- Ronneberger, O., Fischer, P., and Brox, T. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pp. 234–241, 2015. doi: 10.1007/978-3-319-24574-4_28.
- Siper, M., Khalifa, A., and Togelius, J. Path of destruction: Learning an iterative level generator using a small dataset. In *IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 337–343, 2022.
- Sudhakaran, S., Grbic, D., Li, S., Katona, A., Najarro, E., Glanois, C., and Risi, S. Growing 3D artefacts and functional machines with neural cellular automata. In *Artificial Life Conference Proceedings (ALIFE)*, 2021. doi: 10.1162/isal.a_00451.
- Summerville, A., Behrooz, M., Mateas, M., and Jhala, A. The learning of zelda: Data-driven learning of level topology. In *Proceedings of the FDG Workshop on Procedural Content Generation*, 2015.
- Summerville, A. J., Snodgrass, S., Mateas, M., and Ontañón, S. The VGLC: The video game level corpus. In *Proceedings of the 7th Workshop on Procedural Content Generation*, 2016. arXiv:1606.07487.
- Zhang, H., Fontaine, M., Hoover, A., Togelius, J., Dilkina, B., and Nikolaidis, S. Video game level repair via mixed integer linear programming. In *Proc. AAAI Conf. on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, volume 16, pp. 151–158, 2020.

Appendix

A. Degradation under stochastic damage. Figure 1 reports the RSR of the main model against the extent of the erasure damage, on the test split, averaged over eight training seeds. The collapse is concentrated in a narrow band: the rate falls from 0.358 when 5% of the room is destroyed to 0.031 at 20%, and approaches zero beyond that. Above roughly one fifth of the room there is not enough surviving local context for the rule to reconstruct from.

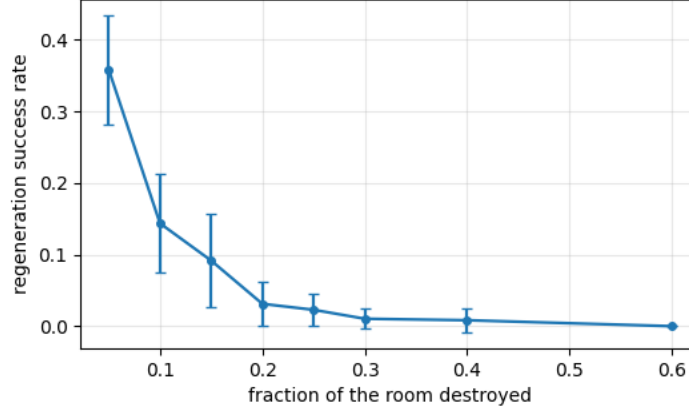


Figure 1. RSR against the fraction of the room destroyed, test split, mean and standard deviation over eight training seeds.

B. Ablations. All ablations are reported on the validation split, since we used them to understand the behaviour of the model rather than to produce final numbers. Each configuration varies a single hyperparameter with respect to the main model and uses three seeds, except the Laplacian and the BFS variant, which use eight paired seeds. Table 2 reports the four axes. Capacity, unroll length, update probability and MLP width all leave access repair (B1) essentially unchanged, with deviations of the same order as the means, while several of them improve stability (A0) and the two conditions where the damage is locally consistent (A3 and B3). Doubling the unroll length is the clearest case: 128 steps give the worst access repair of the whole study, 0.033, and the highest value comes from the shortest horizon, which is consistent with every additional step re-applying the locally plausible completion.

The Laplacian is worth a note. Its B1 value at eight seeds is 0.127 ± 0.152 against 0.090 ± 0.066 for the baseline, a spread larger than the mean, and it degrades A3 from 0.524 to 0.284. At three seeds the two arms are ordered the other way round, 0.044 against 0.122, so the sign of the comparison depends on how many seeds one looks at. With deviations of this size there is no effect to report, only noise, and this is why the two comparisons the conclusions rest on use eight paired seeds rather than three.

C. Full per-condition results. Table 3 reports the main model on every damage condition of the suite, with both metrics. A1 is the contiguous erasure and A2 the tile flip, at a selection of the extents we evaluated; B2 deletes a segment of the perimeter wall and B4 the cells whose removal breaks the main walkable component, which only applies to the 8 test rooms that have such cells.

Two observations follow. The tile flip is much harder than the erasure at the same extent, 0.035 against 0.358 at 5%: writing a wrong tile is worse than leaving a hole, since the model has to recognise that the tile is wrong before correcting it. And B4, despite removing the cells that hold the walkable area together, is among the easiest conditions at 0.734, because those cells are floor surrounded by floor and the locally plausible completion is the correct one, exactly as in B3.

The contrast between the two columns is why we do not rely on tile accuracy: on B1 the rooms are 98.9% correct and only 10% playable. B2 is a special case, since damage to impassable structure cannot break connectivity by construction: the destroyed cells decode to void, which is not walkable either, so that condition is a fidelity test rather than a topological one.

D. Validating the metric. We check the metric against rooms built by hand, where we know the expected verdict in advance. A room compared with itself must pass; walling an access, turning it into floor, or placing a wall in front of it must fail; adding a monster on the floor, which is walkable, must pass. The case that motivates the relative criterion is a

Table 2. Ablations on the validation split, RSR. Rows in italics are the main model, repeated for reference.

Configuration	A0 none	A3 random	B1 access	B3 isolation
hidden = 8	0.883	0.713	0.217	0.767
<i>hidden = 12</i>	<i>0.967</i>	<i>0.524</i>	<i>0.090</i>	<i>0.713</i>
hidden = 16	0.967	0.417	0.083	0.628
hidden = 24	1.000	0.503	0.067	0.594
unroll ≤ 64	0.967	0.517	0.228	0.700
<i>unroll ≤ 96</i>	<i>0.967</i>	<i>0.524</i>	<i>0.090</i>	<i>0.713</i>
unroll ≤ 128	0.983	0.526	0.033	0.750
update prob. 0.3	0.939	0.537	0.028	0.622
update prob. 0.7	0.994	0.562	0.178	0.756
MLP width 64	0.889	0.447	0.150	0.517
MLP width 256	0.989	0.649	0.178	0.739
with Laplacian	0.940	0.284	0.127	0.677
global channel	0.954	0.360	0.256	0.567

Table 3. Main model on the test split, mean over eight training seeds. B4 is evaluated only on the rooms where it applies.

Condition	Extent	Rooms	RSR	Tile accuracy
A0 none	—	20	0.973	1.000
A1 erasure	0.05	20	0.358	0.984
A1 erasure	0.20	20	0.031	0.913
A1 erasure	0.60	20	0.000	0.615
A2 tile flip	0.05	20	0.035	0.969
A2 tile flip	0.20	20	0.000	0.857
A3 random	K cells	20	0.482	0.995
B1 access	K cells	20	0.100	0.989
B2 wall	5 cells	20	0.819	0.997
B3 isolation	K cells	20	0.675	0.997
B4 articulation	1 cell	8	0.734	0.998

spiral of blocks around a central stair, a layout that occurs several times in the corpus: the stair is unreachable by a naive search even in the pristine room, because the blocks are pushable and the corpus does not say so. Under our criterion the pristine room passes, since the repaired room only has to reproduce the original grouping, and a model that opens the wall and connects the stair fails, so it cannot game the metric by improving the topology. These cases run with the test suite of the repository.

E. The do-nothing baseline. A model that leaves the state untouched scores an RSR of essentially zero on all the topological conditions, so every point above zero is due to the model. This control is what exposed two defects during the project. In the first version a destroyed cell was decoded through an argmax over the visible channels, which on an all-zero vector returns the first index of the alphabet, floor: every destroyed cell silently became walkable and the do-nothing baseline scored up to 0.88. In the second, the criterion compared only the grouping of the cells, so an access turned into floor left the grouping unchanged and passed. Both are the reason the metric decodes empty cells as void and requires the accesses to match.

F. Reproducibility. Inference is stochastic, since the update mask is drawn at every step during evaluation as well. Before we fixed its seed, three evaluations of the same checkpoint on the same split gave B1 values of 0.133, 0.083 and 0.150, a spread larger than half the value itself. All the numbers reported in this paper come from the deterministic version.